

Developer Portfolio Static React Rebuild

Md. Jahid Hasan Raihan — Software Engineer

DOCUMENT VERSION

2.0

SUPERSEDES

PORTFOLIO_PRD.pdf (v1.1, Django + DRF)

HOSTING

Vercel (static)

DATE

1 October 2026

TARGET STACK

React 18 · Vite · Tailwind — no backend

PROJECTS

6 (PromptCanvas removed)

How to use this document. It is written to be the only input needed. Open an empty folder, hand this over, and the result should match the existing site: same content, same design, same behaviour — but served as static files with no API call, no database, and no cold start. Every piece of copy the site displays is reproduced verbatim in §6, so nothing has to be recovered from the old codebase.

1. **Why v2 exists** — the problem with the current deployment
2. **Scope** — what changes, what must not
3. **Architecture** — static data model, file layout, routing
4. **Content authenticity rules** — binding constraints
5. **Owner profile** — identity, education, skills, links
6. **Projects** — all six, with full case study copy
7. **New project source material** — SellFlow BD and ServoraBD
8. **Design system** — palette, type, motion
9. **Pages and components** — what to build
10. **Assets** — images, résumé, optimisation
11. **Quality bar** — accessibility, performance, SEO
12. **Testing**
13. **Deployment to Vercel**
14. **Build order** — recommended sequence
15. **Open items**

01 Why v2 exists

1.1 The problem

The current portfolio is a Django REST API on Render plus a React client. It works, but Render's free tier stops the backend after fifteen idle minutes. The next visitor waits roughly fifty seconds while it boots — during which the page renders but stays empty, because every piece of content arrives from the API.

Measured on the live site: **2.1 s** once warm, **~50 s** cold. A recruiter arriving from a link is almost always the cold case, and a blank page for the better part of a minute reads as broken rather than slow.

1.2 The decision

CORE CHANGE

Remove the backend entirely. Content moves from a database into JavaScript modules bundled at build time, so the first paint already contains every project, skill and image reference. Deployed to Vercel's CDN, nothing sleeps and nothing is fetched.

ASPECT	V1 (CURRENT)	V2 (THIS DOCUMENT)
Content source	PostgreSQL via REST API	JS modules in <code>src/data/</code>
Cold start	~50 s	none
Warm load	~2.1 s	< 1 s target
Hosting	Render web service + static + Neon	Vercel static only
Cost	\$0, or \$7/mo to stop sleeping	\$0, no sleep
Editing content	Django admin	Edit a file, commit, push
Contact form	POST to DRF, stored in DB	Third-party endpoint — see §9.6

1.3 What is given up

- **The admin CMS.** Content edits become code edits. Acceptable: the owner is the only editor and already works in the repository.
- **Server-side contact storage.** Replaced by a third-party form endpoint (§9.6).
- **The Django codebase as a portfolio artefact.** Mitigated: the five Django projects it showcases are themselves the evidence, and the v1 repository remains public.

02 Scope

2.1 Must not change

PRESERVE EXACTLY

Visual design, palette, typography, motion, page structure, every word of existing case study copy, all screenshots, all YouTube links, all repo links, the command palette, the architecture diagrams, dark/light theming, and the accessibility and reduced-motion guarantees. A visitor who knew the old site should notice only that it loads faster and has different projects.

2.2 Changes

#	CHANGE	DETAIL
C1	Remove the backend	No Django, no API, no database. Content is bundled.
C2	Drop PromptCanvas	Remove the project, its route, images and diagram entirely.
C3	Add SellFlow BD	New case study, position 1. Source material in §7.1.
C4	Add ServoraBD	New case study, position 2. Source material in §7.2.
C5	Add three live demo links	SellFlow BD, ServoraBD, MicroMart. The other three keep no live button.
C6	New profile photo	<code>images/raihan2.jpg</code> replaces the current one.
C7	Deploy to Vercel	Replaces the Render pair.

2.3 Non-goals

- No blog, no analytics dashboard, no AI assistant.
- No headless CMS. Content in the repository is the point.
- No server-side rendering. A static SPA is sufficient at this size.
- No redesign. §8 documents the existing system to be reproduced.

3.1 Stack

LAYER	CHOICE	WHY
Framework	React 18	Same as v1; components port directly
Build	Vite 6	Fast, first-class on Vercel
Styling	Tailwind CSS 3	The design system is already expressed in it
Motion	Framer Motion 11	Existing animations depend on it
Routing	React Router 7	Same routes as v1
Icons	Lucide React	Already used throughout
Tests	Vitest + Testing Library	Existing tests port unchanged

Explicitly not used: Next.js, any backend, any database, any state library. React's own hooks are enough.

3.2 The data layer

Everything the API used to return becomes a typed JavaScript module. Images are imported so Vite fingerprints and optimises them; a raw string path would bypass the build.

```
src/data/
├─ profile.js      // identity, hero copy, about, links, résumé
├─ projects.js     // all six, full case studies, imported images
├─ skills.js       // six categories with levels
├─ education.js    // degree, coursework
└─ index.js        // re-exports
```

Shape of one project, matching the v1 API exactly so components need no changes:

```
// src/data/projects.js
import micromartCover from '@assets/projects/micromart/cover.jpg'
import micromartShot1 from '@assets/projects/micromart/01.jpg'

export const projects = [
  {
    slug: 'sellflowbd',
    title: 'SellFlow BD',
    subtitle: '...',
    summary: '...',
    accentLabel: 'SaaS Platform',
    projectType: 'web_app',
    status: 'completed',
    isSolo: true,
    order: 1,
    featured: true,

    problem: '...', // blank lines separate paragraphs
    solution: '...',
    architecture: '...',
    challenges: '...',
    lessons: '...',

    technologies: ['Python', 'Django', ...],
    features: [{ title: '...', description: '...' }],

    githubUrl: 'https://github.com/jhraihaan/SellFlowBD',
    videoUrl: 'https://youtu.be/4FBawFRC9EY',
    liveUrl: 'https://sellflow-web.onrender.com',

    coverImage: sellflowCover,
    images: [{ src: shot1, caption: '...', alt: '...' }],
  },
]
```

CAMEL CASE

v1 served snake_case because DRF did. v2 uses camelCase throughout. When porting a component, rename `cover_image` → `coverImage`, `live_url` → `liveUrl`, `accent_label` → `accentLabel`, `video_url` → `videoUrl`, `github_url` → `githubUrl`, `level_display` → `levelDisplay`, `alt_text` → `alt`, `date_range` → `dateRange`.

3.3 Replacing the fetch layer

v1 routed every read through `useApi` → `services/api.js`. Both disappear. Components import data directly and render it on the first pass, which removes all loading and error states from content rendering.

```
// v1
const { data: projects, loading, error } = useApi(() => api.getProjects())

// v2
import { projects } from '@data'
```

Keep the skeleton components: they still serve image loading and the contact form's pending state. Delete `ErrorState` only if nothing references it after the port.

3.4 File layout

```
portfolio-v2/
├─ public/
│  └─ favicon.svg
│  └─ robots.txt
│  └─ sitemap.xml
│  └─ resume.pdf           // served directly, not imported
├─ src/
│  └─ assets/
│     └─ profile/raihan.jpg
│     └─ projects/<slug>/cover.jpg, 01.jpg, 02.jpg ...
│  └─ components/         // ported unchanged from v1
│  └─ sections/           // home page blocks
│  └─ pages/              // one per route
│  └─ layouts/            // Header, Footer
│  └─ hooks/              // useTheme, useTypewriter, ...
│  └─ data/               // §3.2
│  └─ styles/index.css
│  └─ App.jsx
│  └─ main.jsx
├─ index.html
├─ tailwind.config.js
├─ vite.config.js
├─ vercel.json            // SPA rewrite — see §13.2
└─ package.json
```

3.5 Routes

/	Home
/projects	Index, filterable by technology
/projects/:slug	Case study × 6
/about	About, education, skills
/contact	Contact form
*	404

Client-side routing needs a rewrite rule on the host, or a refreshed case study URL 404s. See §13.2 — this caught the v1 deployment out.

04 Content authenticity rules

These override every other consideration, including making the owner look more impressive.

NEVER INVENT

Jobs · companies · internships · clients · awards · certifications · user counts · revenue · statistics · years of experience · project features · technologies · achievements · testimonials.

Missing information is marked [CONTENT NEEDED] and raised with the owner. Authenticity outranks impressiveness.

EXCLUDED	REASON
AI / ML skills section	Owner is at the learning stage. No RAG, embeddings, vector databases, LangChain, agents or fine-tuning as skills.
"Currently learning AI" line	Drafted once and explicitly removed at the owner's request.
Experience section	No professional history exists.
Achievements section	Nothing significant to display.
CGPA	Recorded privately (3.30); never displayed.
LeetCode count	Footer link only. The number invites comparison.
Skill percentages	Named levels only — a bar implies precision that does not exist.
Hero stat counters	Removed in v1 at the owner's request. Do not reintroduce.
AI attribution	No Claude/AI co-author trailer in commits, PRs, comments or the site. The owner is sole author.

4.1 Design failure modes to guard against

FAILURE	COUNTERMEASURE
Arrogance	No superlatives. Facts only.
"Just a student"	Presented as a software engineer. The degree is a credential, not a qualifier. <i>Highest-priority guard.</i>
Generic template	Custom design, architecture diagrams, named skill levels.
Over-designed	No particles, cursor trails or entry animations.
Unfinished	Empty fields hide their section. No "coming soon", no dead buttons.

4.2 The hiding rule

STRUCTURAL

Any optional field that is empty removes its element entirely rather than rendering a placeholder. A project without `liveUrl` shows no demo button; without `videoUrl`, no walkthrough section; a blank case study field omits that numbered section and the numbering closes up. This is what lets the site ship while parts are still missing.

05 Owner profile

Every value below is supplied by the owner and must be reproduced verbatim.

5.1 Identity

Full name	Md. Jahid Hasan Raihan
Display name	Md. Jahid Hasan Raihan
Title	Software Engineer
Location	Dhaka, Bangladesh — shown publicly
Email	jahidhr05@gmail.com
Availability	Open to remote, hybrid, and onsite roles
GitHub	https://github.com/jhraihan
LinkedIn	https://www.linkedin.com/in/md-jahid-hasan-raihan-745023393
LeetCode	https://leetcode.com/u/Jahid_Hasan_Raihan/ — footer link, no count

5.2 Hero copy — verbatim

Tagline	I build complete, production-shaped web applications.
Intro	Full-stack software engineer working across Django, Django REST Framework, React, and PostgreSQL. I build complete systems — the interface, the API behind it, and the database underneath.
Meta description	Md. Jahid Hasan Raihan — software engineer in Dhaka, Bangladesh. Full-stack applications built with Django, Django REST Framework, React, and MySQL.

Typewriter phrases

The hero cycles these five. Each names something actually built, and the list deliberately spans both ends of the stack:

```
REST APIs with Django & DRF
React interfaces that stay fast
role-based access control
relational database schemas
JWT authentication flows
```

5.3 About — verbatim

Short version, used on the home page:

I build full-stack web applications with Django and React. What interests me is the whole system: how a request travels from the browser through the API into the database and back, how authentication holds it together, and how the schema shapes what is possible.

Full version, used on /about:

I am a software engineer based in Dhaka, Bangladesh, completing a BSc in Computer Science and Engineering at Independent University, Bangladesh.

My programming started with my degree. I chose Computer Science because the field genuinely interested me, and coding turned out to be the part I could not put down. Most of what I know beyond coursework I taught myself, through online courses and by building things that broke until they did not.

Python came first, then data structures and algorithms, which is still where I go to sharpen how I think about problems.

Django came next and stayed — partly because Python already felt natural, and partly because it let me build entire applications rather than fragments. From there: Django REST Framework for APIs, then databases in earnest with MySQL, SQLite, and PostgreSQL, and React on the frontend.

What I care about is complete, working systems. Every project I have built is a full application rather than an isolated feature: role-based access control, JWT authentication, relational schema design, and a decoupled React frontend talking to a REST API. I would rather understand why something works than just get it working.

5.4 Education

Degree	BSc in Computer Science and Engineering
Institution	Independent University, Bangladesh
Location	Dhaka, Bangladesh
Dates	2022 — 2026 · Final year — expected December 2026
CGPA	3.30 — never displayed
Coursework	Data Structures · Algorithms · Database Management Systems · Operating Systems · Computer Networks · Object-Oriented Programming · Software Engineering

5.5 Skills

Levels are named, never numeric. Within a category the strongest come first. Featured five, surfaced most prominently: **Django** · **Django REST Framework** · **React** · **MySQL** · **Python**.

CATEGORY	CORE	STRONG	WORKING	FAMILIAR
Languages	Python	SQL, HTML, CSS	JavaScript	Java
Backend	Django, Django REST Framework, Django ORM	REST API design, Authentication (JWT, session, OAuth)	—	FastAPI, Node.js
Frontend	—	React, React Router, Fetch API	Tailwind CSS, Context API, Vite, Bootstrap	Next.js
Databases	—	MySQL, PostgreSQL, SQLite, Database design and normalisation	Query optimisation	—
DevOps and Deployment	—	Git, GitHub, Environment management	Linux, Vercel / Netlify, Railway / Render / PythonAnywhere	Nginx, GitHub Actions, AWS
Tools	—	VS Code, Postman, Git CLI, AI coding tools	pgAdmin / MySQL Workbench	Figma

06 Projects

Six projects, all solo, all complete. Display order below is the required order. Copy for the four carried over is reproduced verbatim — do not rewrite it.

#	PROJECT	LABEL	LIVE DEMO	STATE
1	SellFlow BD	SaaS Platform	yes	new — write copy
2	ServoraBD	Marketplace	yes	new — write copy
3	MicroMart	E-commerce	yes	copy below
4	EduFlow	EdTech	—	copy below
5	IntelliChat	AI Application	—	copy below
6	MediDesk	Healthcare	—	copy below

REMOVED

PromptCanvas is dropped from v2. Delete its data entry, route, images and architecture-diagram definition. Do not leave a redirect.

1. SellFlow BD

NEW

Slug / route	/projects/sellflowbd
Order	1
Label	SaaS Platform
Subtitle	Multi-tenant order and delivery platform for F-commerce sellers
GitHub	https://github.com/jhraihan/SellFlowBD
YouTube	https://youtu.be/4FBawFRC9EY
Live demo	https://sellflow-web.onrender.com
Stack	Python · JavaScript · React · Vite · Django · Django REST Framework · REST API · JWT · PostgreSQL · SQLite · GitHub Actions

Case study copy

TO BE WRITTEN AT BUILD TIME

Draw problem, approach, architecture, hardest problem and lessons from the project's own README, which the owner supplied in full. Section 6 of this document records the source material. Write in the same register as the existing five case studies: factual, first person, no superlatives.

Screenshots

5 PNGs in `images/Sell-flow/`. The clearest title shot becomes the cover; the rest form the gallery.

2. ServoraBD

NEW

Slug / route	/projects/servorabd
Order	2
Label	Marketplace
Subtitle	Local service marketplace built around a computed trust score
GitHub	https://github.com/jhraihan/ServoraBD
YouTube	https://youtu.be/P6uvjq-dDbA
Live demo	https://servorabd-web.onrender.com
Stack	Python · JavaScript · React · Vite · Django · Django REST Framework · REST API · JWT · PostgreSQL · Playwright

Case study copy

TO BE WRITTEN AT BUILD TIME

Draw problem, approach, architecture, hardest problem and lessons from the project's own README, which the owner supplied in full. Section 6 of this document records the source material. Write in the same register as the existing five case studies: factual, first person, no superlatives.

Screenshots

4 PNGs in `images/servora/`. The clearest title shot becomes the cover; the rest form the gallery.

3. MicroMart

Slug / route	/projects/micromart
Order	3
Label	E-commerce
Subtitle	Multi-role e-commerce platform with integrated payments
GitHub	https://github.com/jhraihan/MicroMart
YouTube	https://youtu.be/6ujWk1MaXg0
Live demo	https://micromart-17ws.onrender.com
Stack	Python · JavaScript · React · Django · Django REST Framework · REST API · JWT · MySQL · SSLCommerz

Summary

A fully decoupled e-commerce platform serving three distinct user types — customers, sellers, and administrators — with product discovery, cart and checkout, payment processing through SSLCommerz, order tracking, and seller and inventory management.

01 · The problem

A working e-commerce system is not one application but several overlapping ones. A customer browsing and buying, a seller managing a storefront and inventory, and an administrator overseeing the marketplace each need a different view of the same data, with different permissions over it. I built MicroMart to work through that problem end to end rather than build a storefront in isolation.

02 · The approach

A decoupled architecture: a React single-page application on the frontend, a Django and Django REST Framework API on the backend, and MySQL underneath. Authentication uses JWT, with the token identifying which of the three user types is making a request and what that user is permitted to do. Payments run through SSLCommerz.

03 · Features (15)

- **Registration and authentication** — JWT-based authentication across three user types.
- **Product search** — Search across the product catalogue.
- **Voice search** — Spoken queries as an alternative to typing.
- **Shopping cart** — Cart management through to checkout.
- **Payment processing** — Checkout through the SSLCommerz payment gateway.
- **Product reviews** — Customer reviews on purchased products.
- **Order tracking** — Order status visible to customer and seller.
- **Store management** — Sellers manage their own storefront.
- **Product and inventory management** — Catalogue and stock levels per seller.
- **Sales analytics** — Sales figures for sellers and administrators.
- **Pricing management** — Seller-controlled product pricing.
- **User management** — Administrative control over platform accounts.
- **Complaint management** — Customer complaints routed for administrative review.
- **Fraud monitoring** — Administrative monitoring for suspicious activity.
- **Seller verification** — Verification step before a seller can trade.

04 · Architecture

The React SPA holds no server state of its own; every read and write goes through the REST API. A request carries a JWT, which the backend verifies before resolving the user's type and permissions. Views are scoped so that a seller reaches only their own store, products, and orders, while an administrator sees across the marketplace. Checkout is the most involved path. The cart is validated against current inventory and pricing, an order is created in a pending state, and the user is handed off to SSLCommerz. The gateway's response determines whether the order is confirmed, inventory is decremented, and the order enters the tracking flow — or whether it is abandoned and the reservation released.

05 · Hardest problem

The hardest part was the complete order and payment workflow. Taking a payment is straightforward in isolation; keeping an order, its inventory, and its payment state consistent is not. A user can abandon a payment mid-flow, a gateway callback can arrive late or twice, and inventory must not be decremented for an order that was never paid for. Getting the states and their transitions right — so that no path leaves the system in a half-finished state — took the most thought of anything in the project.

06 · What I learned

That the interesting part of a feature is usually its failure cases. The happy path through checkout took a fraction of the time; everything after it — abandoned payments, duplicate callbacks, stock that must not be committed prematurely — was the actual work. I also learned how much a clear set of order states simplifies the code around them: once the transitions were explicit, most of the conditional logic scattered through the views disappeared.

Cover

screenshot-2026-09-07-221100.jpg

Gallery (4)

- screenshot-2026-09-07-221111.jpg — Product catalogue and category browsing
- screenshot-2026-09-07-221122.jpg — Product detail and purchase flow
- screenshot-2026-09-07-223935.jpg — Cart and checkout
- screenshot-2026-09-07-223946.jpg — Seller dashboard

4. EduFlow

Slug / route	/projects/eduflow
Order	4
Label	EdTech
Subtitle	Role-based learning management system
GitHub	https://github.com/jhraihaan/Edu-Flow
YouTube	https://youtu.be/yTR0k1KN5f4
Live demo	none — button hidden
Stack	Python · JavaScript · React · Vite · Tailwind CSS · Context API · Django · Django REST Framework · REST API · JWT

Summary

A learning management system with separate Admin, Teacher, and Student roles, covering courses, lessons, assignments, enrolments, submissions, and results. Built with a Django REST Framework backend and a React frontend, with client state handled by native React hooks rather than an external state library.

01 · The problem

A learning platform is defined by who is allowed to do what. A teacher creating an assignment, a student submitting to it, and an administrator overseeing both act on the same records with very different rights. EduFlow was an exercise in getting that permission model right rather than bolting roles onto an application built without them.

02 · The approach

Role-based access control with three distinct roles, enforced on the backend and reflected in the interface so users are never shown controls they cannot use. Authentication is JWT-based, with user lookup by phone number rather than email. The frontend is React with Vite and Tailwind CSS, kept deliberately free of heavy state dependencies.

03 · Features (8)

- **Role-based access control** — Distinct permissions and interface for Admin, Teacher, and Student.
- **Phone-based authentication** — JWT authentication with user lookup by phone number.
- **Course and lesson management** — Full CRUD for courses and their lessons.
- **Assignments and submissions** — Teachers set assignments; students submit against them.
- **Enrolment management** — Students enrolled onto courses.
- **Results** — Recorded and viewable per student.
- **Server-driven notifications** — Interface feedback driven by API responses.
- **Confirmation dialogs** — Destructive actions confirmed before they run.

04 · Architecture

The Django REST Framework backend exposes CRUD endpoints for courses, lessons, assignments, enrolments, submissions, and results. Permissions are enforced at the API layer: the role carried by the authenticated user determines both which records are returned and which operations are allowed on them. On the frontend, client state is managed with native React hooks — `useState`, `useEffect`, and `React Context` — with no external state management library. Server responses drive banner notifications, and destructive actions route through a shared confirmation dialog.

05 · Hardest problem

Keeping one set of permission rules authoritative. Three roles acting on the same records means every endpoint has to answer two questions rather than one: may this user perform this action, and which records may they perform it on. Answering only the first leaves a teacher able to reach another teacher's course simply by changing an id in the URL. The rule I settled on was that the interface never decides access. The API scopes every queryset by the authenticated user's role, and the frontend hides controls purely as a convenience. Hiding a button is a design choice; scoping a queryset is the actual boundary.

06 · What I learned

How much a project benefits from deciding the permission model before writing the first view. Roles added afterwards tend to become scattered conditionals; roles designed in stay in one place. I also deliberately built the frontend with nothing but `useState`, `useEffect`, and `Context`, to find out where that genuinely stops being enough. For an application of this size it never did — which was worth knowing firsthand rather than assuming.

Cover

[screenshot-2026-09-11-230511.jpg](#)

Gallery (4)

- [screenshot-2026-09-11-230523.jpg](#) — Course overview
- [screenshot-2026-09-11-230535.jpg](#) — Assignments and submissions
- [screenshot-2026-09-11-230549.jpg](#) — Results and grading
- [screenshot-2026-09-11-230601.jpg](#) — Administration

5. IntelliChat

Slug / route	/projects/intellichat
Order	5
Label	AI Application
Subtitle	Streaming AI chat application with persistent conversations
GitHub	https://github.com/jhraihan/Intelli-Chat
YouTube	https://youtu.be/D6H_cSP2K68
Live demo	none — button hidden
Stack	Python · JavaScript · React · Tailwind CSS · Django · Django REST Framework · Server-Sent Events · PostgreSQL · Redis · Google Gemini API

Summary

A chat application built on the Google Gemini API, streaming responses token by token over Server-Sent Events. Conversations persist in PostgreSQL so a session survives a page reload, and responses render as Markdown with syntax-highlighted code blocks.

01 · The problem

A chat interface that waits for a complete model response before showing anything feels broken, even when it is working. The response has to arrive progressively, and it has to survive a page reload. Both requirements change how the backend is built.

02 · The approach

Django streams the model's response to the browser over Server-Sent Events rather than returning a single completed reply. Conversations and messages are persisted in PostgreSQL, so a user can reload and continue where they left off. Responses are rendered as Markdown with syntax highlighting and a copy-code control.

03 · Features (5)

- **Streaming responses** — Model output streamed token by token over Server-Sent Events.
- **Persistent conversations** — Conversations and messages stored in PostgreSQL and restored on reload.
- **Markdown rendering** — Headings, lists, tables, links, and fenced code blocks.
- **Syntax highlighting** — Highlighted code blocks with a copy-code control.
- **Authentication** — Register, sign in, and sign out, with sessions surviving reload.

04 · Architecture

React sends a request to POST /api/chat/. Django calls the Gemini API and streams the result back as a sequence of Server-Sent Events: a meta event first, then repeated delta events carrying generated text as it arrives, and finally either a done or an error event to close the stream. Conversations and their messages are written to PostgreSQL, which is what allows a reloaded page to restore prior context. Users register, sign in, and sign out, and the session survives a reload.

05 · Hardest problem

Streaming forced me to give up the request/response shape I was used to. A normal Django view builds a complete response and returns it; here the connection has to stay open while text arrives in fragments, each one forwarded to the browser immediately. That created a second problem: a stream can end in more than one way. It can complete, it can fail partway through, and the client can disconnect while the model is still generating. I settled on an explicit event protocol — a meta event, repeated delta events, and a terminating done or error — so the frontend always knows which of those happened instead of inferring it from a stream that simply stopped.

06 · What I learned

That a persistence layer changes what an interface can promise. Once conversations and messages were stored in PostgreSQL rather than held in memory, a page reload stopped being a data-loss event, and the frontend could be

much simpler as a result. More generally, this project taught me to design the protocol between client and server first. Once the event sequence was decided, both sides became straightforward to write.

Cover

screenshot-2026-09-09-220646.jpg

Gallery (3)

- screenshot-2026-09-09-220636.jpg — Sign-in with demo access
- screenshot-2026-09-09-220709.jpg — Streaming response with syntax-highlighted code
- screenshot-2026-09-09-221030.jpg — Conversation history

6. MediDesk

Slug / route	/projects/medidesk
Order	6
Label	Healthcare
Subtitle	Hospital management system with four-role access control
GitHub	https://github.com/jhraihan/Medi-Desk
YouTube	https://youtu.be/PQhEb0xRwpY
Live demo	none — button hidden
Stack	Python · JavaScript · React · Django · Django REST Framework · REST API · JWT

Summary

A hospital operations system covering patients, doctors, appointments, prescriptions, and billing, with JWT authentication and role-based access across Admin, Doctor, Patient, and Receptionist.

01 · The problem

Hospital workflows are a permissions problem before they are a software problem. A receptionist books appointments but must not read clinical notes; a doctor issues prescriptions but does not manage billing; a patient sees only their own records. MediDesk models those boundaries across four roles.

02 · The approach

JWT authentication with access to each module restricted by role. Appointments, prescriptions, patient and doctor records, billing, and a medicine catalogue each expose only the operations a given role is permitted to perform.

03 · Features (7)

- **Role-based authentication** — JWT authentication with module access by role across four user types.
- **Appointment management** — Book, approve, complete, or cancel, with filtering by doctor, patient, and date.
- **Prescription system** — Digital prescriptions linking multiple medicines with dosage and duration.
- **Doctor and patient portals** — Full CRUD for patient records and doctor profiles.
- **Doctor availability** — Real-time availability toggle affecting bookings.
- **Billing and invoicing** — Patient bills with tracked totals and payment status.
- **Medicine inventory** — Searchable catalogue of hospital medicines and pharmacy registry.

04 · Architecture

Appointments move through an explicit lifecycle — booked, approved, completed, or cancelled — with filtering by doctor, patient, and date. Prescriptions link a consultation to multiple medicines, each with its own dosage and duration, so a prescription is a structured record rather than free text. Billing generates patient invoices with tracked totals and payment status. Doctor availability is a live toggle, which is what makes the booking flow reflect real capacity.

05 · Hardest problem

Modelling a prescription properly. Storing it as text would have been quick, but a prescription is genuinely structured data: one consultation links to several medicines, and each of those carries its own dosage and duration. Getting that relation right was what made the medicine catalogue and the prescription history useful rather than decorative. Four roles also made access control harder than the three-role projects. A receptionist books appointments but has no business reading clinical records; a doctor issues prescriptions but does not manage billing. Each module had to be reasoned about separately rather than governed by one blanket rule.

06 · What I learned

That the schema decides what the application can do later. Because prescriptions were modelled as real relations rather than free text, features like searching the medicine catalogue and linking dosages came almost for free —

appointment as a lifecycle rather than a row. Booked, approved, completed, and cancelled are distinct states with rules about which transitions are legal, and naming them explicitly kept the logic contained.

Cover

screenshot-2026-09-14-175313.jpg

Gallery (3)

- screenshot-2026-09-14-175326.jpg — Appointments and scheduling
- screenshot-2026-09-14-175338.jpg — Patient records
- screenshot-2026-09-14-175353.jpg — Prescriptions and billing

07 New project source material

Both new case studies must be written from the owner's own README text, summarised below. Write in the same register as the existing four: factual, first person, no superlatives, and no claim that is not in the source.

7.1 SellFlow BD

What it is: a multi-tenant SaaS platform for Bangladeshi Facebook and Instagram sellers. One seller signs up, creates a store, invites staff, and runs order entry, confirmation, inventory, courier booking, delivery tracking, COD reconciliation, returns and profit from a single dashboard.

The problem

Thousands of Bangladeshi businesses sell entirely through Facebook and Instagram. Orders arrive as Messenger chats, comments and phone calls; almost everything ships cash on delivery through a third-party courier. There is no storefront, no checkout and no system of record, so the seller tracks orders in a notebook and discovers months later that the business is not profitable.

Cash on delivery is the fact that makes the domain hard — the seller ships goods before being paid, which creates every requirement below.

PROBLEM	HOW IT IS HANDLED
Orders scattered across DMs, comments and calls	One order book with search, filters and a full status timeline
Fake and unconfirmed orders waste delivery money	Phone confirmation with logged call outcomes; stock committed only once confirmed
The same customer orders twice on two channels	Duplicate detection warns on a recent open order with overlapping products, with explicit override
Repeat offenders who refuse delivery	Automatic customer risk scoring from delivery and return history, plus a blacklist
Overselling stock already promised	Separate <code>on_hand</code> and <code>reserved</code> counts, so availability is always truthful
Courier money arrives in a bulk spreadsheet	Upload statement, match against shipments, review a preview, then commit; shortfalls surfaced as a number
Returns quietly destroy the margin	Returns capture forward delivery cost, return charge and written-off stock, feeding into profit
Revenue looks healthy but profit is unknown	Net profit after COGS, delivery cost, return loss and operating expenses

Architecture — the three pieces carrying most of the weight

- **Tenant isolation, enforced in three layers.** A base manager returning nothing when no store is resolved; a mixin resolving the tenant from the user's own memberships; a view mixin scoping every query and injecting the store on writes. Cross-tenant access returns 404, never 403, so the API never confirms another tenant's record exists.
- **The order state machine** declares legal transitions as data and attaches stock side effects to them, inside a transaction with a row lock and an immutable audit trail.
- **Inventory is an append-only ledger.** Every movement records its resulting balances, and concurrent reservations are serialised with `select_for_update()` so the last unit in stock cannot be sold twice.

Fourteen domain apps, each shaped the same way: `models.py` for structure, `services.py` for business logic, thin views handling only HTTP. The same `create_order()` runs for the dashboard, the public order form and the API alike.

Access is capability-based: five roles (owner, manager, order staff, delivery staff, accountant) map to named capabilities checked on every request.

Candidate "hardest problem"

COD reconciliation is the strongest choice. A courier statement is uploaded as CSV and becomes a draft; nothing settles until confirmed. Rows land in four buckets — matched, amount mismatch, unmatched, already settled. Committing settles²¹

only matched rows; mismatches are skipped unless explicitly opted into, because a silent mismatch is money quietly lost. Column names vary between couriers, so the parser accepts several spellings and strips currency symbols.

Alternatives, if a different angle reads better: multi-tenant isolation failing closed, or serialising concurrent stock reservations.

Other facts available

- Revenue recognised on *Delivered*, never on order creation — a pending COD order is not revenue.
- Money is always `DecimalField(max_digits=12, decimal_places=2)`; a test walks every analytics endpoint and fails if any value returns as a float.
- Courier credentials encrypted at rest with Fernet and never returned by the API.
- Every courier sits behind one adapter interface; manual, Pathao and Steadfast ship today.
- No Celery or Redis: scheduled work runs from a GitHub Actions schedule calling a protected endpoint every 15 minutes.
- Phone numbers normalise to `+8801XXXXXXXXX` on save.
- Order numbers come from an atomic per-store counter, never `COUNT(*) + 1`.

7.2 ServoraBD

What it is: a marketplace connecting customers in Dhaka with verified electricians, plumbers, AC technicians and other tradespeople. The owner's own framing: *"The point of this project is not the booking flow — that is a well-understood problem. It is the trust model."*

The problem with star ratings

A provider with one 5-star review outranks a provider with two hundred jobs averaging 4.8. Ratings cluster so tightly at 4.5–5.0 that they carry almost no information, and they only measure jobs that were completed — a provider who accepts ten bookings and abandons eight can hold a perfect score.

USE CAREFULLY

The README contrasts two provider profiles (Kamal 84.2, Shakib 53.5) to show the difference. These are **illustrative examples the system produces**, not real users. If used on the site, frame them that way — never as customer data.

The trust score — six factors

FACTOR	WEIGHT	CAPTURES
Verification depth	20%	Has this person proven who they are?
Job volume	15%	Is there enough evidence to judge them?
Completion reliability	20%	When they commit, do they finish?
Cancellation discipline	15%	How often do they break a commitment?
Responsiveness	10%	How long does a customer wait?
Review quality	20%	What do verified customers say?

The three techniques that make it work

- **Bayesian smoothing** on completion rate and reviews, so low-volume providers regress toward the platform mean instead of scoring 100 on a single job.
- **Damage-weighted, time-decayed cancellations** — cancelling an hour before costs 4× what cancelling two days ahead costs, and old cancellations fade with a 180-day half-life.
- **Logarithmic volume** saturating at 100 jobs, so farming trivial jobs has sharply diminishing returns.

Penalties for upheld disputes apply *after* the weighted sum, so one serious incident cannot be diluted by strong performance elsewhere.

Candidate "hardest problem"

Designing a score that resists gaming. Each technique above answers a specific attack: a single glowing review, a well-timed cancellation, or a pile of trivial jobs. The anti-gaming design is the engineering story.

Verification figures — attribute carefully

ATTRIBUTION

These come from the owner's own test runs and seeded data, not production traffic. Present them as measured locally, never as live usage.

- 587 backend tests; 47 frontend unit and component tests.
- Playwright runs both golden paths plus a WCAG 2.1 AA scan of every page, Chromium at a 360 px viewport.
- Provider search p95 **100 ms** across 10,018 seeded providers (target 400 ms).
- Nightly trust recompute for 10,018 providers in **175 s** (target 15 min).
- Landing page LCP **1.58 s** on Slow 4G with 4× CPU throttling (target 2.5 s).
- axe-core reports no WCAG 2.1 AA violations; pip-audit and npm audit report zero known vulnerabilities.
- The trust engine reproduces both PRD worked examples exactly (84.19 and 53.48).

The security review fixed a rate-limit bypass through forged `X-Forwarded-For` headers, stopped accepting HTML files renamed to `.jpg`, added an audit failing the suite if any route omits its permission check, and upgraded to Django 5.2 LTS.

Deployment status — resolve before writing

OPEN QUESTION

The README says "*The site is not deployed yet.*" The owner supplied `servorabd-web.onrender.com` as a live link. Confirm which is current before publishing a demo button — a dead demo link is worse than none.

08 Design system

Reproduce exactly. Every value was contrast-checked; do not substitute by eye.

8.1 Palette

Defined once as CSS custom properties on `:root` and `:root.light`, consumed as Tailwind tokens. Nothing hardcodes a colour.

TOKEN	DARK (DEFAULT)	LIGHT	ROLE
--canvas	8 10 14	250 251 253	Page background
--surface	16 20 27	255 255 255	Cards
--elevated	24 30 40	241 245 249	Inputs, tags
--line	38 46 60	222 229 238	Default border
--line-strong	58 70 90	194 205 219	Hover border
--ink	241 245 249	12 17 26	Primary text
--ink-muted	154 166 184	71 85 105	Body text
--ink-faint	106 118 138	117 130 150	Metadata
--accent	34 211 238	14 116 144	Accent
--accent-hover	103 232 249	21 94 117	Hover
--on-accent	8 10 14	255 255 255	Text on accent
--violet	167 139 250	109 40 217	Category tint
--green	52 211 153	4 120 87	Category tint
--amber	251 191 36	180 83 9	Category tint
--pink	244 114 182	190 24 93	Category tint

WHY DARK TEXT ON THE ACCENT

Cyan-400 reaches **10.96:1** on the dark canvas. White text on that same cyan reaches only 3.68:1 — AA-large, not enough for button labels. So buttons use `--on-accent` (near-black) on dark, and the light theme drops to cyan-700 where white works at 5.36:1.

8.2 Typography

TOKEN	SIZE	USE
hero	clamp(3rem, 7.5vw, 6.5rem) / 0.98 / -0.04em	Hero name only
display-lg	clamp(2.75rem, 6vw, 5rem)	Reserved
display	clamp(2.25rem, 4.5vw, 3.5rem)	Page titles
heading-lg	clamp(1.75rem, 3vw, 2.5rem)	Section headings
heading	clamp(1.375rem, 2vw, 1.75rem)	Subsections
body	1.0625rem / 1.7	Prose

Inter for reading (400–800); JetBrains Mono for metadata, eyebrows, tags, diagram nodes and keyboard hints.

NAMING TRAP — DO NOT REPEAT

The hero size was first named `display-x1`. Tailwind silently generated nothing because it prefix-collides with the existing `display` key, and the heading fell back to 16 px. It is named `hero` for that reason. Avoid font-size keys that prefix another key. Note also that Tailwind config changes need a Vite restart — they are not hot-reloaded.

8.3 Motion

EFFECT	BEHAVIOUR
Scroll reveal	Fade and 16 px rise on entering the viewport
Typewriter	Hero cycles five phrases with a blinking caret and a panning gradient
Magnetic buttons	Primary actions drift toward the cursor; transform applied to the node, no re-render
Spotlight cards	Cursor-following radial glow via CSS custom properties, not state
Ambient glows	Three slow drifting blurs behind the hero, in accent / violet / green
Floating portrait	6 s vertical float with corner brackets
Card hover	Accent border glow, cover image scales 1.04, gradient ring fades in
Scroll progress	Hairline accent bar under the header
Skeletons	Shimmer placeholders matching real content shape

NON-NEGOTIABLE MOTION RULE

Content must never depend on an animation to become visible. The `Reveal` component enforces this two ways: reduced motion renders a plain element, and a 900 ms timer reveals regardless — so a missed intersection cannot strand real content at zero opacity.

This exists because it was violated once. Case study sections rendered into the DOM but stayed permanently invisible: `whileInView` never fired inside a collapsed grid child. Every section of every case study was affected.

8.4 Layout tokens

- Container `max-width: 75rem`, prose `44rem`.
- Section padding `clamp(4.5rem, 10vw, 8.5rem)`.
- Card radius `0.875rem`.
- Grid backdrop 64 px; small variant 28 px; dot field 20 px.
- Decorative glows must sit inside a clipped container — a negative z-index escapes the parent's `overflow-hidden` and causes horizontal scroll on mobile.

09 Pages and components

9.1 Home

#	SECTION	CONTENT
1	Hero	Availability pill, name at hero scale, title with terminal glyph, typewriter, intro, actions, floating portrait. No stat counters.
2	Featured projects	All six cards in a responsive grid
3	Systems, not screens	Four engineering principles — copy in §9.2
4	What I work with	Six skill categories, level legend, coursework
5	About preview	Short bio and education card
6	Contact CTA	Closing call to action with direct email

9.2 "Systems, not screens" — verbatim

Eyebrow *How I build*; description: "The same four decisions shape every project below. They are the reason these are applications rather than demos."

TITLE	BODY	TINT
Decoupled by default	A React client that holds no server state, talking to a REST API that owns all of it. Every project is built this way, which keeps the contract between frontend and backend explicit.	accent
Permissions at the API	Access rules live on the server, not in the interface. A hidden button is a design choice; a scoped queryset is a security boundary. MicroMart, EduFlow, and MediDesk all enforce roles at the API layer.	violet
Schema before features	The database shape decides what the application can do later. I design relations and constraints first, then build against them — across PostgreSQL, MySQL, and SQLite.	green
State machines over flags	Orders, appointments, and submissions each move through defined states. Getting those transitions right is what stops a system ending up in a half-finished condition.	amber

With SellFlow BD added, both the permissions and state-machine entries could name it too — it is the strongest example of each. Optional.

9.3 Case study page

Up to six numbered sections, numbered only across those that render:

```
01 The problem
02 The approach
03 Features      (two columns, check glyphs)
04 Architecture  (request-flow diagram + prose)
05 Hardest problem
06 What I learned
```

Then, conditionally: a YouTube embed if `videoUrl` is set, and a screenshot gallery if images exist. A sticky sidebar lists the stack.

Images use `fit="contain"` in the gallery and on the cover, because the screenshots are wide desktop captures that lose their edges under `cover`. Project cards keep `cover` — a card needs a filled thumbnail.

9.4 Architecture diagrams

Inline SVG, one per project, defined in `components/ArchitectureDiagram.jsx` . Terminal-window chrome with a caption, colour-coded nodes, an optional dashed branch for an external service. Horizontal on wide screens, stacked with downward connectors on narrow ones so no node is clipped. A project with no definition renders nothing.

PROJECT	FLOW	BRANCH
selfflowbd	React → Django + DRF (tenant-scoped) → PostgreSQL	Courier APIs (Pathao / Steadfast)
servorabd	React → Django + DRF → PostgreSQL	Trust engine (scheduled recompute)
micromart	React SPA → Django + DRF (JWT verified) → MySQL	SSLCommerz
eduflow	React + Vite → DRF (permission layer) → Database	—
intellichat	React (SSE listener) → Django (stream relay) → PostgreSQL	Gemini API — animated connectors
medidesk	React (4 role views) → Django + DRF (JWT + RBAC) → Database	—

Node tones: client = accent, api = violet, database = green, payment branch = amber, model branch = violet.

9.5 Command palette

`Cmd/Ctrl + K` from anywhere. Groups results into Navigation, Case studies, Actions and Links; built from the data module so a new project appears without touching it. Arrows navigate, Enter selects, Escape closes — Escape bound at the document so it works regardless of focus. Guard `scrollIntoView` : it is absent in some environments and throws in jsdom.

9.6 Contact form — the one real decision

Without a backend the form needs a third-party endpoint. Recommended: **Formspree** free tier (50 submissions/month) or **Web3Forms** (250/month, no account).

```
const res = await fetch('https://formspree.io/f/<id>', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json', Accept: 'application/json' },
  body: JSON.stringify({ name, email, subject, message }),
})
```

Preserve from v1: client-side validation with per-field errors, `aria-invalid` and `aria-describedby` , the honeypot `website` field, loading state, success state with "Send another message", and a plain `mailto:` fallback in the sidebar so the form is never the only route.

The endpoint id is not a secret — it is public by design and rate-limited by the provider.

9.7 Components to port unchanged

- `Reveal` — with its 900 ms guarantee
 - `Spotlight` — cursor glow
 - `Magnetic` — button drift
 - `ImageFrame` — placeholder + `fit` prop
 - `ProjectCard`
 - `ArchitectureDiagram`
- `CommandPalette`
 - `SectionHeading` , `Skeleton` , `ThemeToggle`
 - `Header` with scroll progress, `Footer`
 - Hooks: `useTheme` , `useTypewriter` , `useCountUp` , `useScrollProgress` , `usePrefersReducedMotion` , `useDocumentTitle`

Delete: `useApi` , `services/api.js` , and `ErrorState` if unreferenced.

10 Assets

10.1 Sources

ASSET	SOURCE	COUNT
Profile photo	images/raihan2.jpg (614 KB)	1
SellFlow BD	images/Sell-flow/	5
ServoraBD	images/servora/	4
MicroMart	images/micro-mart/	5
EduFlow	images/edu-flow/	5
IntelliChat	images/intelli-chat/	4
MediDesk	images/medi-desk/	4
Résumé	cv/cv_part1.pdf (89 KB)	1

PromptCanvas images are not carried over.

10.2 Optimisation

Raw PNG screenshots are roughly ten times larger than needed. Convert once at import time:

Screenshots	Max width 1600 px, progressive JPEG, quality 84. Observed: 1017 KB → 129 KB.
Profile photo	Square centre crop, 800×800, quality 88. Observed: 989 KB → 96 KB.
Location	src/assets/ and imported, so Vite fingerprints them. Not public/ .
Résumé	public/resume.pdf — referenced by path, never imported.

10.3 Covers

The cover is the screenshot showing the product's name most clearly — usually the landing or sign-in screen. For the four carried-over projects the existing covers are already chosen; \$6 names each file.

11 Quality bar

11.1 Accessibility — all verified in v1, must hold

- Semantic HTML with a `main` landmark
- Skip link as the first tab stop
- Exactly one `h1` per page
- `lang` set on the document
- Every image carries alt text
- Every button and link has an accessible name
- Visible focus ring, keyboard users only
- Form errors linked by `aria-describedby`
- Filter buttons expose `aria-pressed`
- Result regions `aria-live="polite"`
- All motion off under `prefers-reduced-motion`
- No horizontal scroll at 320 px

11.2 Performance targets

MEASURE	V1 ACTUAL	V2 TARGET
Bundle, gzipped	≈124 KB	≤ 140 KB
First contentful paint	~2.1 s warm, ~50 s cold	< 1 s, always
Content visible	after API response	first paint
Lighthouse performance	not measured	≥ 95

Vendor chunks split as in v1: react, motion, app. Images lazy by default; hero and first two cards eager.

11.3 SEO

- Per-page title and meta description via `useDocumentTitle` .
- Open Graph and Twitter Card tags.
- JSON-LD `Person` schema — job title, location, institution, `sameAs` profile links.
- `sitemap.xml` covering all nine routes, plus `robots.txt` .
- **Set the canonical URL to the real Vercel domain at launch.** v1 shipped with `example.com` for weeks.
- Create `/og-image.png` . v1 referenced one that never existed, so link previews were imageless.

12 Testing

Vitest plus Testing Library. Port the v1 suite and extend it. Several tests deliberately encode the content rules from §4, so a change that breaks a promise fails the suite rather than shipping.

SUITE	ASSERTS
<code>ImageFrame</code>	Renders an image; substitutes the placeholder without one; honours <code>loading</code> and <code>fit</code>
<code>ProjectCard</code>	Content, case study link, GitHub link, live demo hidden without a URL , technology collapsing
<code>ArchitectureDiagram</code>	Known flows, external branch, branch omitted when absent, nothing for an unknown slug
<code>CommandPalette</code>	Closed state, listing, filtering, empty result, Escape, links omitted when absent
<code>data (new)</code>	Six projects; PromptCanvas absent; exactly three have <code>liveUrl</code> ; every project has a cover, a GitHub URL and a video URL; no skill exposes a numeric level
<code>Contact (new)</code>	Validation errors, honeypot present, success state

Manual: every route in both themes with no console errors; mobile at 320/390/412 px; reduced-motion; the contact form end to end.

13 Deployment to Vercel

13.1 Setup

```
# Vercel → Add New → Project → import the repository
Framework preset  Vite          # auto-detected
Build command     npm run build
Output directory  dist
Install command   npm ci
```

No environment variables are required. If a Formspree id is used it may be inlined — it is public by design.

13.2 SPA rewrite — required

THIS BROKE V1

Without a rewrite, `/projects/micromart` opened directly returns 404. Navigating inside the site works, so it is easy to miss — but every shared link is the failing case. Vercel reads `vercel.json`; a Netlify `_redirects` file is ignored.

```
// vercel.json
{
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
}
```

13.3 Launch checklist

- All six case studies open directly, not only by clicking
- Three live demo buttons appear; the other three show none
- All six video embeds play
- Every screenshot loads; none broken
- Résumé downloads
- Contact form submits and mail arrives
- Light/dark toggle persists across reload
- `Cmd/Ctrl + K` opens and navigates
- No console errors on any route
- Canonical URL and OG tags point at the real domain
- `og-image.png` exists
- Sitemap lists all nine routes
- No horizontal scroll at 320 px
- Lighthouse ≥ 95 performance, 100 accessibility

13.4 Custom domain

Roughly \$12/year. `jahidhasan.dev` reads considerably better than a `.vercel.app` subdomain. After adding it, update the canonical link, both `og:url` tags, and regenerate the sitemap.

14 Build order

#	STEP	VERIFY BEFORE MOVING ON
1	Scaffold Vite + React + Tailwind; copy <code>index.css</code> and <code>tailwind.config.js</code> verbatim	Dev server runs; accent token resolves
2	Optimise and place all assets (§10)	Sizes match the targets
3	Write <code>src/data/</code> — profile, skills, education, four carried-over projects	Copy is byte-identical to §5–6
4	Port components and hooks; delete the fetch layer	No import of <code>useApi</code> remains
5	Build Home, Projects, ProjectDetail, About, Contact, 404	Every route renders, no console errors
6	Write the two new case studies from §7	No claim absent from the source
7	Add their diagram definitions	Both render, and stack on mobile
8	Wire the contact form	A real submission arrives
9	SEO, sitemap, OG image	Real domain, not a placeholder
10	Tests	All pass, including the new data suite
11	Deploy with <code>vercel.json</code>	Direct case study URL returns 200

15 Open items

#	ITEM	NEEDED FROM
O1	ServoraBD live status. README says not deployed; a live URL was supplied. Confirm before publishing the demo button.	Owner
O2	New CV. The owner mentioned replacing it, but <code>cv/</code> still holds only the September file. Supply a newer PDF or confirm the current one stands.	Owner
O3	Public email. The site publishes <code>jahidhr05@gmail.com</code> ; <code>hello.jahidhr@gmail.com</code> is also in use. Confirm which belongs on the site.	Owner
O4	Contact endpoint. Formspree or Web3Forms; the owner creates the account and supplies the id.	Owner
O5	Custom domain. Optional but recommended.	Owner
O6	Skills refresh. The table in §5.5 predates SelfFlow BD and ServoraBD. Both used PostgreSQL heavily and ServoraBD used Playwright; consider whether any level has genuinely moved. Do not inflate.	Owner

Portfolio v2 PRD — Md. Jahid Hasan Raihan · 1 October 2026. Every factual claim originates from the owner's own statements or from the verified state of the v1 site. Where information was unavailable it is listed in §15 rather than assumed.